数据库课程设计报告

StarX-Car 网约车系统

数据库设计与应用实现报告

年级：待填写

班级：待填写

姓名：待填写

学号：待填写

指导教师：待填写

完成日期：2026 年 9 月 27 日

目录

摘要

一、系统概述

二、需求分析

三、系统设计

四、系统实现

五、系统测试

六、总结与展望

参考资料

说明：本报告以项目仓库中的数据库初始化脚本、接口说明和各端 README 为设计依据；实时交互统一按 WebSocket 事件推送口径描述。

摘要

StarX-Car 是面向乘客、司机和管理员三类角色的网约车系统，覆盖账号认证、车辆管理、即时与预约叫车、订单状态流转、计价规则、司机位置、余额支付、评价、投诉售后和运营管理等业务。系统以后端数据库为核心，采用 MySQL 5.6.50 保存账户、角色资料、车辆、订单、计价规则、账单、位置快照及审计数据；后端使用 Java 21、Spring Boot 和 MyBatis-Plus 实现分层业务；乘客端和司机端采用 Flutter Android，管理端采用 Vue 3。

本设计重点解决三类问题：第一，使用订单主表、状态事件表和业务明细表记录可追溯的订单生命周期；第二，通过外键、唯一约束、版本号、事务和触发器保证高并发场景下的数据一致性；第三，将订单状态变化、司机位置、预约派单、费用和售后结果统一抽象为 WebSocket 实时事件，由服务端主动推送给授权客户端，客户端按角色订阅，不依赖反复发起查询。系统中高德相关坐标统一采用 GCJ-02，金额采用数据库定点小数和客户端字符串传输，避免坐标系混用和浮点误差。

关键词：网约车；MySQL；订单状态机；WebSocket；事务一致性；MyBatis-Plus；Flutter Android

一、系统概述

1.1 项目背景与建设目标

移动出行服务同时包含位置、订单、车辆、计价和支付等多种数据。传统的单体式页面容易把乘客流程、司机流程、运营流程和数据库操作混在一起，导致权限边界不清、订单状态难以追踪、费用无法审计。StarX-Car 以数据库课程设计为目标，建立一个具有清晰角色边界和完整订单闭环的网约车数据系统。

系统建设目标是：建立统一的用户和角色模型；实现即时单与预约单的创建、抢单、接驾、服务、完成、取消和异常结束；将计价规则版本与订单绑定，保留费用明细；以最新位置快照支撑司机接驾和行程地图；用余额账户、账本和支付结算事实支撑模拟支付和退款；通过评价、投诉及管理员审计完成服务闭环；以 WebSocket 连接向不同角色推送其有权限接收的实时事件。

1.2 系统范围与角色

角色	主要职责	可访问数据
乘客	注册登录、选择地点、获取报价、即时/预约叫车、取消、查看司机和路线、支付、评价、投诉	本人资料、本人订单、订单司机和车辆展示字段、本人余额和售后记录
司机	注册登录、提交资料和车辆、出车、接单、到达、开始/结束服务、导航、查看收入和评价	本人资料和车辆、可抢订单、本人已接订单、本人位置、收入和收到的评价
管理员	用户管理、司机审核、订单监控、预约运营、计价规则、投诉处理、运营备注和审计	运营范围内的用户、订单、费用、司机位置文本和投诉数据
系统服务	令牌校验、订单状态机、预约调度、计价结算、余额账本、事件广播和审计	按事务执行所需的业务数据，不作为普通用户角色登录

1.3 技术路线与设计原则

层次	选型	设计要点
移动客户端	Flutter 3.44.8 / Dart 3.12.2	乘客端与司机端独立负责角色页面、状态管理和业务流程；共同基础设施由 starx_car_common 提供
管理端	Vue 3 + TypeScript + Vite + Pinia + Axios	面向管理员的订单运营、司机审核、计价和投诉工作台
后端	Java 21 + Spring Boot 3.5.16 + Spring Security 6	按业务域分层，Controller 保持薄，应用服务编排事务，领域层保存状态机和计算规则
持久层	MySQL 5.6.50 + MyBatis-Plus 3.5.7	InnoDB、utf8mb4、外键、索引、触发器、乐观锁版本和定点小数
实时通信	WebSocket + JSON 事件信封	Bearer JWT 鉴权，按用户/订单/司机/运营范围订阅，心跳、重连、序号和最后已知位置恢复
地图服务	高德 Android SDK / Web 服务	POI、定位、逆地理编码和路径规划统一使用 GCJ-02；Key 只放在运行配置中

1.4 系统边界

公共包只提供网络、认证、令牌刷新、安全存储、地图基础能力和角色无关模型，不放入乘客或司机页面、订单状态机、派单、定价、数据库访问及完整业务 Repository。乘客端和司机端分别拥有自己的订单流程；后端集中管理订单、计价、支付和审计；管理端只通过受保护接口和运营事件访问管理数据。

二、需求分析

2.1 功能需求

功能域	业务需求	关键数据
认证与账号	注册、登录、刷新令牌、修改资料和密码、注销；密码不能明文保存	user、passenger、driver、admin、JWT 声明
车辆与司机	司机资料审核、绑定多辆车、切换当前车辆、上线/下线、位置上报	driver、car、driver_car_binding、driver_location
叫车与订单	报价、下单、预约、取消、抢单、到达、身份核对、开始服务、完成或中途结束	ride_order、ride_order_event、order_cancellation
计价	起步价、里程、行程时间、等待时间和到达后取消费分段计价；规则可版本化	pricing_rule、pricing_rule_tier、order_fare_item
支付与售后	余额充值/提现、订单结算、退款、投诉证据和管理员处理	user_balance_account、balance_transaction、order_payment、complaint 系列表
评价	完成订单后七天内一单一评，维护司机聚合评分	order_rating、driver.rating_sum/rating_count/rating
运营管理	订单监控、预约派发、计价规则维护、司机审核、追加	order_operation_note、

功能域	业务需求	关键数据
	备注、写操作审计	admin_operation_log 及关联业务表
实时交互	订单状态、抢单结果、司机位置、预约派单、支付和售后状态主动推送	WebSocket 事件信封、订单事件和最新位置快照

2.2 业务流程分析

即时叫车流程由乘客选择上车点和目的地开始。客户端先校验两点不相同并确认坐标为 GCJ-02，服务端调用高德路径规划，读取当前默认计价规则并返回报价。乘客确认后，服务端在事务中锁定乘客资料、重新规划路线、写入订单和 CREATE 事件。订单进入 PENDING 后，司机端通过授权 WebSocket 订阅收到待抢订单事件，抢单成功的第一个司机通过版本条件更新获得订单。

订单接单后依次经过 ACCEPTED、DRIVER_ARRIVED、IN_SERVICE、COMPLETED 等状态；乘客或司机在合法状态下可以取消，服务中可按规则中途结束。每次状态转换均由状态机校验，并在同一事务中更新订单、写入事件、维护司机工作状态和位置关联，再向相关订阅者广播 ORDER_STATUS_CHANGED。

预约叫车先进入 SCHEDULED，预约调度器到达派单窗口后转为 PENDING，事件广播给符合条件的司机；超出宽限时间仍未接单则由系统取消。预约派发、提前派发和过期清理都保留事件与原因，便于运营追溯。

2.3 非功能需求

指标	要求	实现思路
一致性	订单、司机状态、支付和账本不能出现半成功	事务边界、行级锁/乐观锁、唯一约束、幂等键和状态机
实时性	订单状态和位置变化应在授权客户端建立连接后及时到达	WebSocket 长连接、事件序号、心跳、重连和快照恢复
安全性	不同角色只能读写授权数据，敏感字段不泄露	JWT + Spring Security + @PreAuthorize + 归属校验 + 脱敏字段

指标	要求	实现思路
可追溯性	订单、费用、投诉和管理员操作可回放	ride_order_event、order_fare_item、账本流水和管理员审计
可维护性	业务变化不破坏其他角色和公共基础设施	业务域分层、common 职责边界、版本化计价规则和 DTO
兼容性	面向 Android，适配 MySQL 5.6.50	Flutter FVM 版本固定，SQL 使用 5.6 可用特性，时间精度统一到秒

2.4 订单状态与事件需求

状态	进入条件	允许的下一步	需推送事件
SCHEDULED	预约单创建	PENDING、CANCELLED	RESERVATION_CREATED / ORDER_STATUS_CHANGED
PENDING	即时下单或预约派发	ACCEPTED、CANCELLED	ORDER_AVAILABLE / ORDER_STATUS_CHANGED
ACCEPTED	司机原子抢单	DRIVER_ARRIVED、CANCELLED	ORDER_ASSIGNED / DRIVER_LOCATION_UPDATED
DRIVER_ARRIVED	司机确认到达	IN_SERVICE、CANCELLED	DRIVER_ARRIVED / ORDER_STATUS_CHANGED
IN_SERVICE	尾号核对通过后开始服务	COMPLETED、TERMINATED、CANCELLED	SERVICE_STARTED / FARE_UPDATED
COMPLETED	司机完成服务并结算	评价、投诉、支付售后	ORDER_COMPLETED / PAYMENT_UPDATED
TERMINATED/CANCELLED	异常结束或取消	评价/投诉/退款（按规则）	ORDER_TERMINATED / REFUND_UPDATED

三、系统设计

3.1 总体架构设计

系统采用客户端—后端服务—数据库三层结构，并把地图和图床作为外部适配器。客户端只通过统一 API 和 WebSocket 连接访问后端，不直接访问数据库；后端通过领域服务完成鉴权、订单状态转换、计价、支付和审计，再通过 MyBatis-Plus 写入 MySQL。

层次	组成	职责与依赖方向
乘客端	starx_car_passenger + common	叫车、行程、司机位置和售后；依赖 common 的网络、认证和地图能力
司机端	starx_car_driver + common	听单、抢单、导航、位置上报和服务状态；依赖 common，但不把司机策略放入 common
管理端	StarX-Car-Admin	管理员登录、运营订单、司机审核、计价和投诉；通过 Axios 调用后端
业务服务层	auth、driver、passenger、car、order、pricing、payment、rating、complaint	Controller 接口、应用服务事务编排、领域状态机/计算、Mapper 持久化
实时层	WebSocket Gateway / Event Publisher	连接鉴权、订阅授权、心跳、序号、重连快照和事件广播
数据层	MySQL 5.6.50	业务事实、关系约束、索引、触发器和审计数据

3.2 数据库概念结构设计

系统以 user 为身份根实体。passenger、driver 和 admin 使用与 user 相同的主键，形成一对一角色扩展；driver 通过 driver_car_binding 与 car 构成多对多历史关系，同时用 driver.car_id 指向当前用车。passenger 与 driver 通过 ride_order 发生业务联系，ride_order 再关联计价规则、位置、状态事件、费用明细、支付、取消、评价和投诉。管理员对订单的运营写操作由 admin_operation_log 独立审计。

实体关系	基数	关系说明
user—passenger/driver/admin	1 : 0..1	角色表主键同时是 user 外键，删除和注销策略按角

实体关系	基数	关系说明
		色区分
driver—car	1 ： N（历史绑定）	driver_car_binding 保存绑定/解绑历史，driver.car_id 表示当前车辆
passenger—ride_order	1 ： N	一个乘客可有多条订单，未结束即时单和预约单受业务约束
driver—ride_order	1 ： N	司机抢单成功后关联订单，订单保留车辆快照
ride_order—ride_order_event	1 ： N	记录状态迁移、操作方、时间和备注
pricing_rule—pricing_rule_tier	1 ： N	同一规则按里程、行程时间、等待时间配置连续分段
ride_order—order_fare_item	1 ： N	订单结算产生可审计的分段费用，保存规则版本快照
ride_order—order_payment/order_rating/order_cancellation	1 ： 0..1	支付、评价和异常结束事实分别使用唯一订单约束
complaint—evidence/handle_log	1 ： N	投诉证据和处理过程采用追加式明细

3.2.1 规范化与完整性约束

数据库设计满足第三范式。用户基本资料、角色资料、车辆资料和订单事实分别存放，避免重复保存；计价规则与分段拆分，避免在订单表中固定列出任意数量的阶梯；订单费用明细保存规则和分段快照，使历史订单不依赖当前规则；投诉处理日志和余额流水只追加，避免覆盖历史事实。

完整性约束包括：主键保证实体唯一；外键保证角色、车辆、订单、规则和投诉关联有效；用户名、邮箱、车牌、规则名称版本、订单号、支付号、交易号、投诉号等使用唯一约束；订单评价、支付和异常结束使用一单一条唯一约束；余额流水使用幂等键；手机号脱敏字段由 MySQL 触发器维护，避免 Java 写入遗漏。

3.2.2 主要关系表与字段设计

表名	主键/唯一约束	核心字段与用途
user	id；username、email 唯一	账号、密码哈希、角色、状态、真实姓名、头像和登录时间

表名	主键/唯一约束	核心字段与用途
admin	id 外键	管理员角色扩展，与 user 一对一
passenger	id 外键	手机号、尾号和脱敏展示号
driver	id 外键；car_id 外键	司机证件、审核状态、工作状态、当前车辆和评分聚合
car	id；car_number、车牌拆分唯一	车辆牌照、车型、品牌、颜色、座位和状态
driver_car_binding	id；司机/车辆索引	绑定时间、解绑时间、有效/历史状态
pricing_rule	id；name + version	规则版本、状态、默认标记、起步价和到达取消费
pricing_rule_tier	id；rule/type/sort 唯一	里程、驾车时间和等待时间的连续阶梯
ride_order	id；order_no 唯一	订单类型、角色关联、地点快照、状态、金额和时间线
ride_order_event	id；order_id + created_time 索引	状态前后值、事件类型、操作方和备注
order_fare_item	id；order_id 索引	基础、里程、时间、等待和取消费明细
driver_location	driver_id 主键	每名司机最新 GCJ-02 位置、精度、速度、方向和时间
user_balance_account	id；user_id 唯一	用户余额和乐观锁版本
balance_transaction	id；交易号、幂等键唯一	充值、扣款、司机收入、退款和提现账本
order_payment	id；payment_no、order_id 唯一	应收、实付、退款、司机收入和支付状态
order_cancellation	id；order_id 唯一	取消/中途结束阶段、原因、金额和终点位置
complaint	id；complaint_no 唯一	投诉类型、内容、状态、处理方式和退款金额
complaint_evidence	id；complaint_id + sort_no 唯一	投诉图片证据
complaint_handle_log	id；投诉/时间索引	管理员处理过程和回复
order_rating	id；order_id 唯一	一单一评、评分文字和评价双方
order_operation_note	id；订单/时间索引	管理员追加式运营备注
admin_operation_log	id；订单、管理员和目标索引	取消、提前派发和备注等管理员写操作审计

3.2.3 物理设计与索引

所有业务表使用 InnoDB 和 utf8mb4。金额使用 DECIMAL(10,2) 或 DECIMAL(12,2)，经纬度使用 DECIMAL(10,6)，时间使用 DATETIME；订单主键和明细主键按业务规模使用 BIGINT。订单列表按照乘客、司机状态、订单状态、订单类型和时间建立组合索引；预约调度使用 order_type、status、scheduled_pickup_time 索引；支付和投诉按照参与人、订单和创建时间建立查询索引。

司机位置采用 driver_id 主键的一行最新快照模型，避免第一阶段为了实时展示产生无限增长的轨迹表。位置变更通过 WebSocket 传播，数据库只保留恢复连接所需的最新值；服务端返回 stale 标记时客户端应显示最后已知位置，而不能把旧坐标伪称为当前坐标。

3.3 WebSocket 实时通信设计

本报告将订单、位置、预约、费用、支付、评价和投诉的实时交互统一设计为 WebSocket。客户端建立连接后携带 Bearer JWT 完成握手，服务端根据用户 ID、角色和订单归属生成可订阅范围。common 只负责连接、鉴权、重连、心跳、通用事件信封和连接状态，不负责乘客或司机的订阅策略。

订阅范围	订阅者	推送内容
user/{userId}/orders	对应乘客或司机	本人订单状态、费用、支付、评价和投诉结果
order/{orderNo}	订单乘客、接单司机、授权管理员	订单状态迁移、司机分配、费用变化、取消和异常结束
driver/{driverId}/location	订单乘客、接单司机、授权管理员	最新 GCJ-02 位置、精度、速度、方向和 stale 信息
driver/{driverId}/offers	在线司机本人	待抢订单、预约单派发和订单被他人抢走
admin/orders	active admin	运营订单摘要、预约风险、司机位置摘要和写操作结果

3.3.1 事件信封与连接生命周期

```
{
  "eventId": "evt_20260927_000001",
  "eventType": "ORDER_STATUS_CHANGED",
  "occurredAt": "2026-09-27 10:20:30",
```

```
  "sequence": 42,
  "traceId": "trace_xxx",
  "orderNo": "SX202609270001",
  "data": {"from": "ACCEPTED", "to": "DRIVER_ARRIVED"}
}
```

连接阶段依次为 CONNECTING、AUTHENTICATING、CONNECTED、SUBSCRIBED 和 CLOSED。客户端在 CONNECTED 后提交角色端需要的订阅，服务端校验归属后返回 SUBSCRIBED；正常连接发送心跳，服务端响应 pong。网络中断时客户端使用有限退避重新连接，并携带最后收到的 sequence；服务端优先补发订单事件，司机位置则从 driver_location 返回最新快照。重复事件按 eventId 或 sequence 去重，状态转换仍由服务端数据库版本和状态机保证幂等。

3.3.2 实时事件与数据库事实的对应

事件	触发事务	数据库来源	客户端处理
ORDER_STATUS_CHANGED	创建、抢单、到达、开始、完成、取消、终止	ride_order + ride_order_event	刷新订单状态、时间线和可用操作
DRIVER_LOCATION_UPDATED	司机上报位置	driver_location 最新快照	更新地图标记和路线起点
ORDER_ASSIGNED	原子抢单成功	ride_order.driver_id / car_id + ACCEPT 事件	乘客显示司机资料，其他司机移除待抢项
FARE_UPDATED	报价或结算完成	ride_order + order_fare_item	显示预估/最终费用及分段明细
PAYMENT_UPDATED	余额扣款、收入、退款	order_payment + balance_transaction	更新支付状态、余额和退款结果
COMPLAINT_UPDATED	投诉创建或管理员处理	complaint + complaint_handle_log	展示处理进度、说明和退款金额

3.4 安全设计

认证采用 access token 与 refresh token 分离的 JWT 方案。密码使用 BCrypt 保存，服务端只接受 token_type 正确的令牌；受保护接口通过 Spring Security 角色控制，资源操作还必须执行当前用户归属检查。WebSocket 握手和每次订阅都复用 Bearer 身份，不能因为长连接而绕过订单归属。

手机号由数据库维护尾号和脱敏展示值，司机证件号不返回给无关角色，日志禁止记录密码、令牌、完整手机号、身份证和历史精确轨迹。traceId 贯穿请求、响应和事件信封，用于排查问题但不携带敏感数据。高德 Key、JWT 密钥、数据库密码和签名配置均来自环境变量，不写入源代码和报告。

3.5 事务与并发设计

场景	事务内操作	并发保护
创建订单	锁定乘客资料、校验未结束订单、路线计价、写订单和 CREATE 事件	事务、状态约束和乘客业务前置检查
司机抢单	条件更新订单、写司机/车辆快照、写 ACCEPT 事件、司机变 busy、关联位置	订单 version + status + driver_id 条件更新
完成订单	计算等待/服务时长、更新订单、写费用明细和 COMPLETE 事件、司机恢复 online、清理位置关联	状态机、事务回滚、金额明细总和校验
结算/退款	锁定余额账户、写账户变动和账本、更新支付事实	余额 version、幂等键和退款累计额上限
激活默认规则	锁定旧默认规则、停用旧规则、激活新规则	事务内唯一默认语义和 name/version 唯一键
评价	写评价并原子增加司机 rating_sum/rating_count	order_rating.order_id 唯一约束

四、系统实现

4.1 后端分层实现

后端根包为 org.mannerth.starxcar，按 auth、passenger、driver、car、order、infrastructure、security、web 等业务域组织代码。每个业务域内部划分 api、application、domain、infrastructure、request、response 六类目录，依赖方向为 api -> application -> domain，应用服务通过 Mapper 或适配器访问外部资源。

模块	代表组件	实现内容
认证与安全	AuthController、JwtService、JwtAuthenticationFilter	注册、登录、刷新令牌、BCrypt、当前用户和角色权限

模块	代表组件	实现内容
订单主域	OrderController、OrderLifecycleService、OrderStateMachine	报价、下单、取消、到达、开始服务、完成和中途结束
司机工作	DriverWorkController、DriverDispatchService	上下线、位置快照、待抢订单和原子抢单
计价	PricingRuleService、FareCalculator	规则版本、分段校验、CEIL 计费和费用明细
支付售后	WalletController、OrderSettlementService、ComplaintService	余额、账本、支付、退款、投诉和处理日志
运营	AdminOrderController、AdminPricingRuleController、AdminDriverController	订单监控、预约运营、司机审核、计价规则和管理审计
外部适配	AmapRouteClient、ImageService	高德路线和图片上传，统一异常映射，不泄露上游密钥
实时事件	WebSocket Gateway、Event Publisher（设计层）	将事务结果转换为事件信封，按订阅范围广播并支持重连恢复

4.2 数据库脚本与触发器实现

数据库初始化脚本集中在 StarX-Car-Backend/create_sql.md，包含表创建、索引、外键、触发器和默认计价规则。passenger 与 driver 的手机号触发器在 INSERT/UPDATE 时自动写入 phone_tail 和 display_phone；应用层只提交原始手机号，避免脱敏规则在多端重复实现。

```
CREATE TABLE driver_location (
    driver_id       INT PRIMARY KEY,
    active_order_id BIGINT NULL,
    latitude        DECIMAL(10, 6) NOT NULL,
    longitude       DECIMAL(10, 6) NOT NULL,
    recorded_time   DATETIME NOT NULL,
    received_time   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_driver_location_driver FOREIGN KEY (driver_id) REFERENCES
driver(id)
);
```

ride_order 记录地点快照、估算距离、计费距离、状态和时间线；ride_order_event 记录每次状态转换；order_fare_item 记录规则版本、区间、单位和金额。订单结算时服务端按订单保存的 pricing_rule_id 与 pricing_rule_version 读取历史规则，保证后续调整默认规则不会改变已创建订单的结算依据。

4.3 Flutter 公共模块与角色端实现

模块	实现能力	边界
starx_car_common	Dio ApiClient、统一响应、Bearer 注入、401 刷新、安全存储、GCJ-02 模型、高德地图/定位/搜索/路线、订单展示模型、WebSocket 通用连接器	不包含角色页面、路由、状态管理、完整 Repository、数据库和订单状态机
starx_car_passenger	登录、地点选择、报价/叫车、行程、司机信息、地图、评价、投诉和余额入口	负责乘客订阅哪些订单事件以及如何解释状态
starx_car_driver	登录、资料车辆、上线听单、抢单、导航、位置上报、到达、服务、结算和评价	负责司机 offers、订单和位置上报策略
StarX-Car-Admin	管理员登录、用户管理、司机审核、订单运营、预约运营、计价、投诉和个人中心	负责管理员事件订阅和运营操作交互

4.4 关键业务实现

4.4.1 报价、下单与状态事件

报价请求包含上车点、目的地和订单类型。后端以 GCJ-02 调用高德路线，读取唯一 active + is_default 规则，按分段计算基础费、里程费、行程时间费和等待费。创建订单时服务端再次规划与计价，不接受客户端传入的最终金额，并在订单表保存规则版本快照。事务提交后发送 ORDER_CREATED 和 ORDER_STATUS_CHANGED。

4.4.2 抢单、位置与路线

司机必须审核通过、账号有效、绑定有效车辆并处于 online，才可以订阅待抢订单。位置上报写入 driver_location 最新快照，事件服务将位置变化推送给该司机当前订单的乘客和授权管理员。乘客端在接驾和服务中根据司机位置与目的地重新规划高德路线，司机端根据当前位置和上车点/目的地导航；路线点不作为历史轨迹写入数据库。

4.4.3 完成结算与售后

完成服务时计算实际服务时长和等待时长，写入订单最终金额、费用明细和 COMPLETE 事件，并将司机恢复为 online。余额结算使用 user_balance_account、balance_transaction 和 order_payment 三类事实；退款通过投诉处理或业务规则产生，受已支付金额和累计退款金额限制。客户端收到 PAYMENT_UPDATED 或 COMPLAINT_UPDATED 后更新金额、余额和处理状态。

4.4.4 WebSocket 端到端流程

步骤	服务端动作	事件/响应	客户端动作
1. 建立连接	校验 JWT、记录连接与角色	CONNECTED	common 保存连接状态
2. 请求订阅	校验 user/order/driver/admin 范围	SUBSCRIBED 或 FORBIDDEN	角色端只提交自身业务订阅
3. 业务提交	HTTP 命令接口执行事务	命令结果 + 领域事件	保留命令响应，等待事件作为最终状态来源
4. 事务提交	发布事件，写入 event/outbox（可扩展）	ORDER_STATUS_CHANGED 等	按 eventId/sequence 去重并更新页面
5. 连接恢复	读取最后序号或最新位置快照	RESYNC_COMPLETED	恢复订单状态、地图和未处理提示

4.5 接口与统一响应

HTTP 命令接口继续使用 REST 风格，成功响应由 ApiResponseAdvice 统一包装为 code、message、traceId 和 data；WebSocket 只传输实时事件，不改变业务权限和数据库事务。典型接口包括 POST /api/orders/quote、POST /api/orders、POST /api/orders/{orderNo}/accept、POST /api/orders/{orderNo}/complete、PUT /api/drivers/me/location、POST /api/orders/{orderNo}/rating 和投诉/余额接口。

```
{
  "code": 0,
  "message": "success",
  "traceId": "trace_xxx",
  "data": {}
}
```

错误响应覆盖 400、401、403、404、409 和外部服务不可用等场景，不返回密码、JWT、刷新令牌、SQL 详情或堆栈。WebSocket 错误使用统一 error 事件，包含 code、message、

traceId 和可安全展示的业务原因。

五、系统测试

本章给出课程设计的测试方案和验收用例，重点验证数据库约束、事务边界、角色权限和 WebSocket 事件一致性。测试可使用 Spring Boot 测试配置、MockMvc、服务单元测试、客户端契约测试和经授权的 MySQL 烟测；不依赖生产密钥，不在测试数据中保存真实个人信息。

编号	测试场景	验证步骤	预期结果
T01	乘客注册与登录	提交合法/重复用户名、错误密码和刷新令牌	合法账号获得 access/refresh；重复或错误输入返回统一错误，不泄露密码
T02	角色权限	乘客访问司机接口、司机访问管理员接口、跨用户访问订单	返回 403 或归属不可见的 404；WebSocket 越权订阅被拒绝
T03	报价精度	提交 GCJ-02 两点和边界距离/时间	返回高德距离、时长和 Decimal 费用；坐标精度与范围校验生效
T04	订单创建	创建即时单和预约单，重复创建未结束订单	订单状态正确，CREATE 事件存在，规则版本被保存，冲突返回 409
T05	并发抢单	两个司机同时抢同一 PENDING 订单	只有一个条件更新成功；另一个收到冲突和 ORDER_ASSIGNED 事件
T06	状态机边界	越过非法状态直接开始/完成，重复完成或取消	非法迁移返回 409，不写半成品事件，合法事件顺序可追溯
T07	司机位置	上报有效/超前/错误坐标并检查乘客订阅	只保存最新合法位置，事件推送给授权订阅者，越权用户看不到
T08	WebSocket 重连	断开连接后携带最后 sequence 恢复，重复发送同一事件	订单事件按序补发，位置恢复最新快照，客户端去重不重复展示
T09	预约派发	预约进入派单窗口、管理员提前派发、超时清理	SCHEDULED/PENDING/CANCELLED 状态和事件一致，相关司机收到派发事件
T10	计价规则	创建连续/重叠/空洞分段，切换默认规则	非法规则被拒绝；默认规则切换不影响历史订单版本
T11	完成结算	司机完成服务并模拟余额扣款和收入入账	订单金额等于费用明细总和，账本前后余额正确，司机恢复 online

编号	测试场景	验证步骤	预期结果
T12	退款幂等	重复提交同一退款幂等键或超过实付金额	重复请求不重复扣账，超限返回 409，PAYMENT_UPDATED 只反映一次事实
T13	一单一评	完成订单后提交 1-5 星、重复提交和超七天提交	合法评价写入并原子更新司机评分，重复/超时返回 409
T14	投诉售后	乘客提交投诉，管理员道歉/驳回/部分或全额退款	投诉状态、处理日志和余额变化一致，乘客收到处理事件
T15	触发器与脱敏	新增/修改乘客或司机手机号	phone_tail 和 display_phone 自动同步，应用不需要复制脱敏逻辑
T16	数据库外键与索引	尝试删除有关联记录、检查订单和预约查询计划	外键阻止孤儿记录；主要列表使用对应组合索引
T17	管理员审计	取消订单、提前派发、追加备注	写入 admin_operation_log；普通读取和事件推送不伪造管理员写操作
T18	事件安全	使用失效 JWT、错误角色和过期订阅连接	连接关闭或返回安全错误，traceId 可用于定位且不泄露令牌

5.1 数据库一致性检查清单

检查项	检查内容	合格标准
表结构	表名、字段类型、默认值、字符集和存储引擎	与 create_sql.md 初始化脚本一致，全部业务表为 InnoDB/utf8mb4
关系约束	外键、唯一键、非空字段和一单一条事实	不存在孤儿订单、重复订单号、重复评价或重复支付事实
金额	订单总额、费用明细、支付、退款和账本	金额使用定点小数，费用明细求和与订单金额相等，退款不超过实付
时间	订单状态时间、设备记录时间和服务端接收时间	业务时间精确到秒，设备时间不能绕过服务端时间校验
位置	坐标系、最新快照和订单关联	所有业务坐标为 GCJ-02，终态订单解除位置关联，数据库不产生历史轨迹垃圾
事件	事件类型、顺序、操作者和 traceId	事务提交后事件与数据库事实一致，重连可按序恢复

5.2 性能与可靠性检查

WebSocket 长连接数量、单用户订阅数、事件广播耗时和重连成功率应纳入运行监控；事件消费者应使用有界队列，避免慢客户端阻塞业务事务。数据库侧重点监控订单状态索引、预约窗口索引、账本幂等键冲突和连接池。出现连接中断时，客户端以最新订单快照和最新位置恢复页面，不能把本地缓存直接当作最终事实。

六、总结与展望

本报告完成了 StarX-Car 网约车系统的需求梳理、数据库概念设计、逻辑关系设计、物理实现说明、分层架构和实时通信设计。系统以订单为核心，把角色资料、车辆、计价、位置、费用、支付、评价、投诉和运营审计拆分为职责清晰的关系表；通过外键、唯一约束、索引、触发器、事务和版本号保证数据一致性；通过 WebSocket 事件信封统一表达订单、位置、费用和售后变化，满足乘客、司机和管理员的实时展示需求。

数据库设计还保留了良好的扩展空间。后续可以增加 WebSocket 集群部署中的连接路由和消息代理、事件持久化/补发表、附近司机空间索引、完整轨迹历史及留存清理策略；可以接入第三方支付和对账、司机收入统计、自动派单和动态定价；也可以增加风控、设备指纹、消息通知和更细粒度的管理员审计。所有扩展都应先评估表结构、索引、数据留存和事务边界，再同步更新初始化脚本和接口契约。

在课程设计层面，StarX-Car 的核心收获是将一个复杂移动业务拆解为可验证的数据事实：订单状态由事件记录证明，费用由明细解释，支付由账本对账，位置由快照恢复，角色权限由身份和归属约束。以这些事实为基础，客户端体验和 WebSocket 实时推送都可以在不破坏数据库一致性的前提下演进。

参考资料

[1] StarX-Car-Backend/create_sql.md：数据库初始化脚本、表结构、索引、触发器和默认计价规则。

[2] StarX-Car-Backend/README.md：后端技术栈、已实现能力、接口概览和配置说明。

[3] docs/client-api-contract.md：客户端接口、统一响应、坐标和订单字段约定。

[4] starx_car_common/README.md、starx_car_passenger/README.md、starx_car_driver/README.md：Flutter 公共模块及角色端职责和功能说明。

[5] StarX-Car-Admin/README.md：管理端页面、运营接口和权限说明。

[6] MySQL 5.6 Reference Manual：InnoDB、外键、索引、触发器和事务。

[7] Spring Boot、Spring Security、MyBatis-Plus、Flutter 和高德 Android SDK 官方文档。